

브랜치 전략과 동기화 판단의 기준

- 🌱 가벼운 포인터 브랜치 ✓ 명확한 네이밍 규칙
- ↔ 효율적인 동기화 (Merge/Rebase) 🏗️ 협업 모델 선택
- 🏗️ 협업 모델 선택 규칙 사전 합의

LG Display CH04 | 2026. 04. 10

 LG Display

TITLE Button [협업의 기반, 좋은 브랜치]



- 커밋 가리키는 가벼운 포인터 방식:
폴더 복사 없이 독립적 분리
- 좋은 브랜치의 3가지 조건
 1. 목적 명확
 2. 짧은 수명
 3. 분명한 병합 기준

TITLE Button [명확한 명명 규칙]



- 역할이 보이는 이름 부여 ('feature/user-profile', 'fix/login-timeout')

TITLE Button [수명 관리와 충돌]



- 브랜치 수명 짧게 유지: main과의 격차가 커질수록 병합 충돌 비용 증가

TITLE Button [리뷰 효율 & 안정성]



- main 브랜치 보호 (승인된 PR만 허용), 코드 리뷰 효율성 및 장애 복구 능력 향상

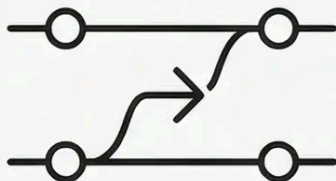
Git 명령어 사용의 명확성 확보: switch vs. checkout

GIT SWITCH

브랜치 전용 명령어 (Git 2.23+): 역할 분리로 명확성 향상
초보자에게 권장: 직관적인 브랜치 작업 (혼란 방지)

```
# 기존 브랜치로 이동
git switch <브랜치명>

# 새 브랜치 생성 후 즉시 이동
git switch -c <새브랜치명>
```



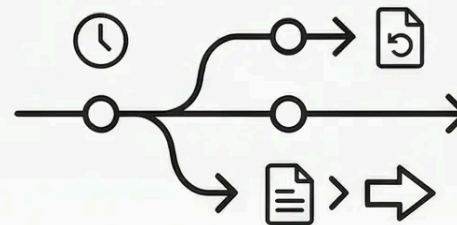
GIT CHECKOUT

다목적 광범위 명령어: 과거 기능 통합
브랜치 이동 외 기능: 파일 복원, 커밋 탐색 등

```
# (기존 방식) 브랜치로 이동
git checkout <브랜치명>

# (기존 방식) 새 브랜치 생성 및 이동
git checkout -b <새브랜치명>

# 특정 시점의 코드 조사 (파일 복원)
git checkout <커밋SHA> -- <파일명>
```



헤드라인 / Headline

Upstream 핵심 개념



Upstream이란?

- 로컬 브랜치가 추적하는 원격 브랜치



`git branch -vv` 명령어

- 로컬 브랜치와 upstream의 연결 상태를 한눈에 출력

```
$ git branch -vv
feature/login 1a2b3c [origin/feature/login]
* main        4d5e6f [origin/main: ahead 2]
develop      7h8i9j [origin/develop: ahead 1]
```

브랜치 동기화 상태 및 조치

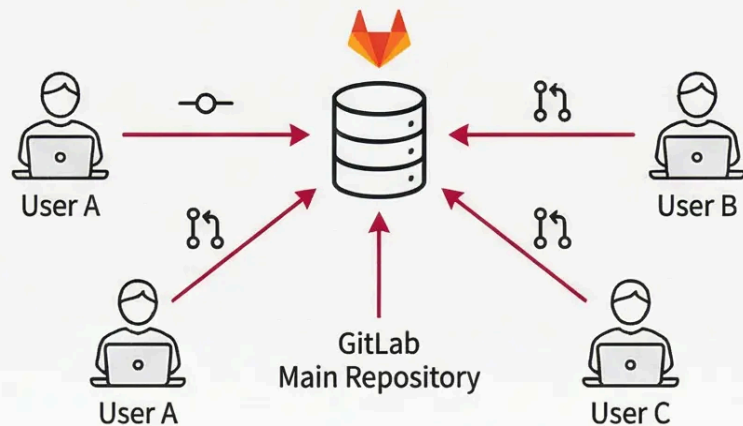
↑	ahead	로컬 커밋이 원격보다 앞서 있어 push가 필요한 상태 ahead 2
↓	behind	원격 커밋이 로컬보다 앞서 있어 pull 또는 fetch가 필요한 상태 behind 1
↔	ahead / behind	로컬과 원격이 동시에 커밋됨 (Forked) • Merge 또는 Rebase 결정 필요 ahead 2, behind 1
🔗	Upstream 최초 설정	Upstream 연결을 최초로 지정할 때 사용 • `git push -u origin 브랜치명`으로 upstream 지정 `git push -u origin 브랜치명`으로 upstream 지정



팀의 성격과 보안 정책에 따른 저장소 모델 선택

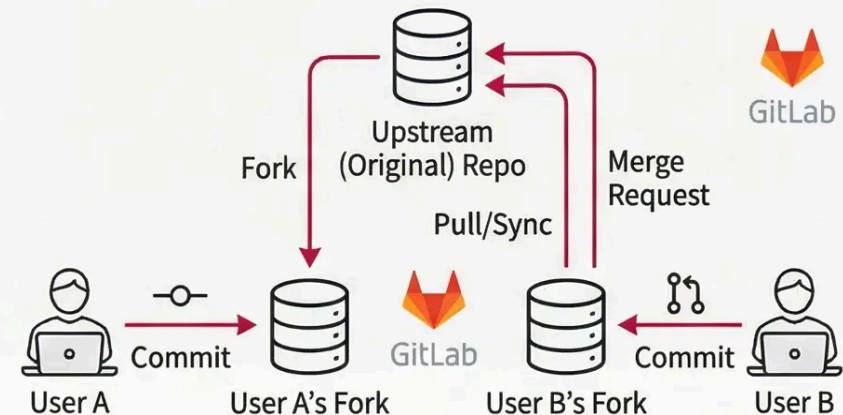
사내 팀 협업의 기본 모델 Shared Repository

- 단일 중앙 저장소에 모든 팀원 직접 접근
- 기능 브랜치 작업 후 Merge Request(MR) 병합
- **보안 권한:** 내부 구성원에게 쓰기 권한 부여
- **팀 운영 정책:** main 브랜치 보호, 승인된 MR 필수



외부 기여자와 오픈소스에 최적화 Fork Model

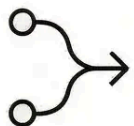
- 원본 저장소를 개인 계정에 복제(Fork)하여 독립 작업
- 원본의 안정성을 유지하며 Pull Request(PR/MR) 제출
- **보안 권한:** 외부 기여자에게 직접 권한 부여 불필요
- **운영 부담:** 다수 포크 저장소의 상태 추적 및 관리 필요



merge와 rebase는 무엇을 바꾸는가 / What do merge and rebase change?

주요 특징 및 활용 기준 비교

MERGE



이력 보존 통합

두 브랜치를 병합하고 병합 커밋 생성,
분기 흐름 그대로 기록.



안전성 및 투명성

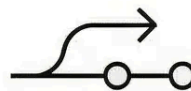
충돌 위험 감소, 협업 시 이전 상태
확인 가능.



팀 권장 전략

초보자가 사용하기 쉬움,
공유 브랜치의 기본 방식.

REBASE



선형 이력 관리

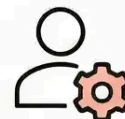
커밋 최신 기준 위로 재배치,
깔끔한 한 줄 이력 정리 (SHA 변경됨).



SHARED

SHA 변경 및 강제 푸시 위험

공유 브랜치에 강제 푸시 시
타인의 기준점 파괴.



개인 브랜치 정리

공유 전 기능 브랜치를 다듬는 데 활용.

Merge와 Rebase 선택의 핵심 기준과 모범 사례



기본 원칙

- 팀 공유 브랜치 (main, develop): merge 사용 (이력 보존)
- 개인 기능 브랜치 (공유 전): rebase 사용 (이력 정리)



공유 후 금지

- 팀원과 공유된 브랜치: rebase + force push 절대 금지
- 협업 기준점 붕괴 및 충돌 유발



수명 및 정리

- 단명 브랜치 권장 (오래된 브랜치는 merge 비용 급증)
- 정리 기준: 병합 후 즉시 삭제, 2주 이상 비활성 브랜치 정리 정책

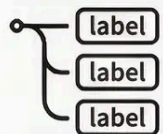


팀 정책 문서화

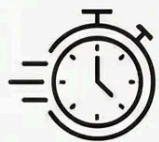
- 팀 정책으로 기준 합의 (초보자 고려)
- merge 기본 전략 및 예외적 rebase 허용 문서화

효율적이고 충돌 없는 협업을 위한 핵심 브랜치 운영 전략

운영 원칙 및 명명 전략



1. 명확한 목적 명시: **prefix 활용** (feature/, fix/, hotfix/)
역할 즉시 파악 및 관리 용이



2. 짧은 수명 브랜치: **main과의 차이 최소화**
대규모 충돌 방지 및 신속한 통합



3. 팀 전체 합의: **일관된 규칙 수립**
검색성 및 추적성 동시 향상

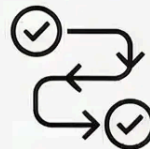
협업 프로세스 및 검증



4. 병합 전 동기화: **fetch로 원격 변경 확인**
`git fetch origin` 습관화로 충돌 감지

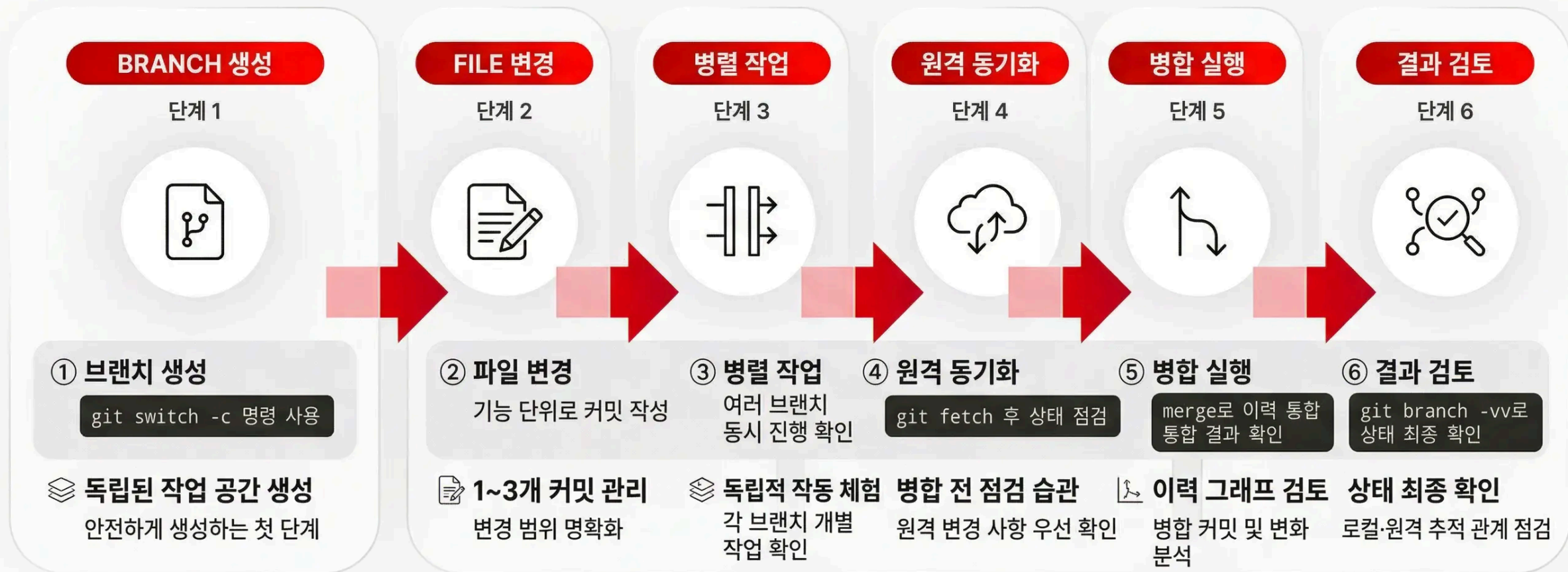


5. 상태 점검: **git branch -vv 활용**
`git branch -vv`
로컬-원격 추적 브랜치 관계 및 동기화 상태 파악



6. 명확한 병합 기준: **충돌 비용 최소화**
목적·수명·기준 삼박자 충족

단계별 실습 절차



명령 흐름과 이력 관리 비교 / Command Flow & History Management

TITLE

주요 특징 비교 / Key Comparison

MERGE 방식 / Merge Method



git fetch 후 merge 실행
(예: git merge origin/main)

별도 merge commit 생성

REBASE 방식 / Rebase Method



git fetch 후 rebase 실행
(예: git rebase origin/main)

커밋 재배포치 & SHA 변경

상세 비교 & 실습 결론 / Detailed Comparison & Conclusion

비교 항목	Merge 방식	Rebase 방식
충돌 발생 시점 및 처리	1회 발생, 한꺼번에 해결	커밋 단위로 반복 발생 가능
Push 후 이력 영향	공유 브랜치에 안전, 일반 push 가능	공유 브랜치에 위험, force push 필요
권장 상황	공유 브랜치 동기화, feature 병합	개인 기능 브랜치 커밋 정리 (깔끔한 이력)

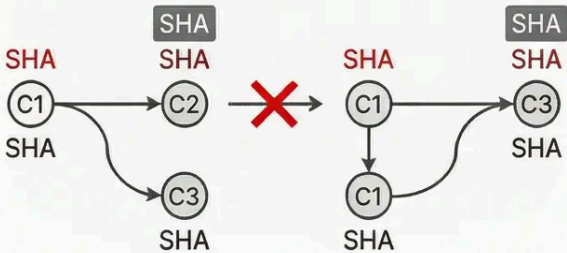
실패 시나리오: rebase 후 push 거부 및 오래된 feature 브랜치 / failure scenario: rebase 후 push 실패와 오래된 feature branch

요약 및 대응 원칙

push 실패 (SHA 변경)



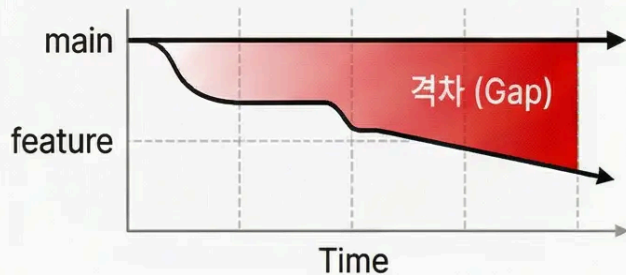
- 커밋 ID 변경으로 로컬과 원격 이력 충돌 충돌 발생
- 거부 메시지: non-fast-forward 오류



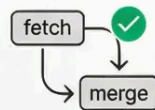
오래된 브랜치 (누적 격차)



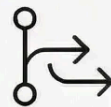
- 오래된 브랜치는 main과 격차 누적 위험
- 브랜치 수명이 길수록 충돌 해결 비용 급증



안전한 협업 및 대응 원칙



- 안전한 동기화 순서:
fetch → merge origin/main
- force push는 공유 브랜치에서 금지
- 공유 브랜치는 merge 우선,
rebase는 개인 브랜치로 제한



Merge Preferred

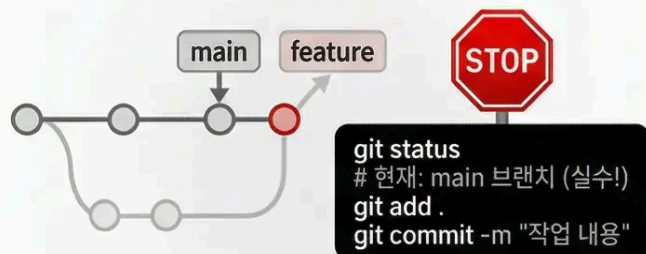


Prohibited Force Push

주요 개발 실수 및 대응 전략

TITLE

잘못된 브랜치 작업

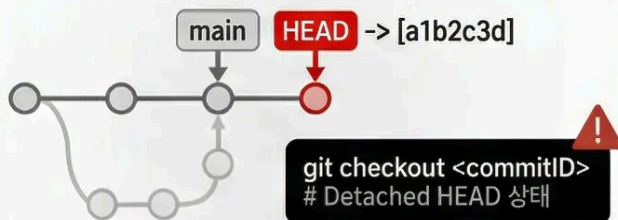


잘못된 브랜치에서
작업

main 브랜치 직접
커밋 시 팀 전체
이력 오염 위험

`git status` 및 `git branch -vv` 로 현재 위치 진단
`git switch <branch>` 로 작업 브랜치 이동

Detached HEAD 발생



Detached HEAD
상태의 징후와 원인

git checkout
<커밋ID> 사용 시
HEAD가 브랜치를 잃음

작업 전 switch 확인, fetch 선행 습관
`git branch <new\_branch>` 로 임시 복구

오래된 브랜치 위험



Stale 브랜치로
인한 충돌 위험

오래된 브랜치는
main과 격차가 커져
충돌 비용 증가

`git fetch --prune` 주기적으로 추적 상태 갱신
`git switch -c` 로 새 브랜치 생성 후 커밋 복원

덜 꼬이게 일하는 branch 운영 규칙



1. 브랜치 본질

- 가벼운 포인터, 목적·수명·기준이 핵심
- 명확한 이름 규칙([code:feature/fix/hotfix])으로 혼선 방지



2. 저장소 모델

- Shared vs. Fork 팀 상황에 맞게 선택
- 내부 팀은 Shared, 외부 기여자 포함 시 Fork 권장



4. 역할별 규칙

- 책임 분리:
Owner·Maintainer·Developer
- main** 직접 푸시 금지, fetch 먼저 확인하는 습관 필수



3. 동기화 전략

- Merge는 안전, Rebase는 이력 정리용
- 공유 브랜치에서 rebase 후 `git push --force` **절대 금지** ❌



5. 다음 장 연결 (1)

- 보호 브랜치, 역할 관리
- CH04 전략이 권한 설정의 토대



6. 다음 장 연결 (2)

- MR 승인 프로세스
- 체계적인 코드 통합을 위한 실질적 토대